Stack และ Queue ด้วย C++ STL

โน้ตนี้อธิบายเรื่อง Stack และ Queue โดยใช้ C++ Standard Template Library (STL) พร้อมคำอธิบายที่ชัดเจนและตัวอย่างโค้ดที่เข้าใจง่าย เหมาะสำหรับการเรียนการสอน

---

1. Stack คืออะไร?

Stack คือโครงสร้างข้อมูลที่ทำงานแบบ
LIFO — Last In, First Out (เข้าหลัง ออกก่อน)
ภาพจำง่ายๆ คือ **กองจาน** — วางจานใบใหม่บนสุด และหยิบออกก็หยิบจากบนสุดก่อนเสมอ
ตัวอย่าง — ใส่ข้อมูลเข้า stack ตามลำดับ

```
push(1) → push(2) → push(3)
```

สถานะของ stack

```
← top (บนสุด)
3
2
1
```

เมื่อ `pop()` ตัวที่ออกมาคือ **3** (ตัวที่ใส่ล่าสุด)

```
← top
2
1
```

---

2. Stack ใน C++ STL

C++ มี `std::stack` ให้ใช้งานได้เลย
สิ่งสำคัญที่ต้องรู้: **stack ไม่ใช่ container ตรงๆ** แต่เป็น **container adaptor**
หมายความว่ามันห่อหุ้ม container อื่นไว้ข้างใน (default คือ `deque`)
การ map ระหว่าง stack operation กับ container ข้างใน

Stack Operation	สิ่งที่เกิดข้างใน
push(x)	push_back(x)
pop()	pop_back()
top()	back()

---

3. Operations ของ Stack

Operation	ความหมาย	Complexity
push(x)	ใส่ x เข้าบนสุด	O(1)
pop()	ลบตัวบนสุดออก	O(1)
top()	อ่านค่าตัวบนสุด (ไม่ลบ)	O(1)
empty()	เช็คว่า stack ว่างไหม	O(1)
size()	จำนวน element ใน stack	O(1)

> ⚠️ **จุดที่มักผิด:** `pop()` ไม่คืนค่า — ต้องอ่านด้วย `top()` ก่อน แล้วค่อย `pop()`

---

4. ตัวอย่างการใช้งาน Stack

In [ ]:

```cpp
#include <iostream>
#include <stack>
using namespace std;

int main() {

    stack<int> s;

    s.push(10);
    s.push(20);
    s.push(30);

    cout << "Top element: " << s.top() << endl;  // 30 — บนสุด

    s.pop();  // ลบ 30 ออก

    cout << "Top after pop: " << s.top() << endl;  // 20

    cout << "Stack size: " << s.size() << endl;  // 2

}
```

5. Pattern ที่ใช้บ่อย: วนลูปจนกว่า Stack จะว่าง

```cpp
while (!s.empty()) {
    cout << s.top() << " ";
    s.pop();
}
```

ผลลัพธ์คือ element ถูกพิมพ์ใน**ลำดับกลับกับที่ใส่เข้าไป** (LIFO)
ตัวอย่าง use case ในการแข่งขัน:

- **Bracket matching** — ตรวจวงเล็บว่าถูกต้องไหม
- **Next Greater Element** — หาตัวที่มากกว่าถัดไป
- **DFS iterative** — แทน recursion

---

6. Queue คืออะไร?

Queue คือโครงสร้างข้อมูลที่ทำงานแบบ
FIFO — First In, First Out (เข้าก่อน ออกก่อน)
ภาพจำง่ายๆ คือ **คิวซื้อของ** — คนที่มาก่อนได้จ่ายเงินก่อนเสมอ
ตัวอย่าง — ใส่ข้อมูลเข้า queue ตามลำดับ

```
push(1) → push(2) → push(3)
```

สถานะของ queue

```
front → 1  2  3 ← back
```

เมื่อ pop() ตัวที่ออกมาคือ **1** (ตัวที่ใส่เข้ามาก่อน)

```
front → 2  3 ← back
```

---

7. Queue ใน C++ STL

C++ มี std::queue ให้ใช้งานได้เลย
Queue ก็เป็น **container adaptor** เช่นกัน (default ใช้ deque ข้างใน)
การ map ระหว่าง queue operation กับ container ข้างใน

Queue Operation	สิ่งที่เกิดข้างใน
push(x)	push_back(x)
pop()	pop_front()
front()	front()
back()	back()

---

8. Operations ของ Queue

Operation	ความหมาย	Complexity
push(x)	ใส่ x เข้าท้ายสุด	O(1)
pop()	ลบตัวหน้าสุดออก	O(1)
front()	อ่านค่าตัวหน้าสุด (ไม่ลบ)	O(1)
back()	อ่านค่าตัวท้ายสุด (ไม่ลบ)	O(1)
empty()	เช็คว่า queue ว่างไหม	O(1)
size()	จำนวน element ใน queue	O(1)

> ⚠️ **จุดที่มักผิด:** pop() ลบตัว **หน้าสุด** (front) ไม่ใช่ท้ายสุด

---

9. ตัวอย่างการใช้งาน Queue

In [ ]:

```cpp
#include <iostream>
#include <queue>
using namespace std;

int main() {

    queue<int> q;

    q.push(10);
    q.push(20);
    q.push(30);

    cout << "Front: " << q.front() << endl;  // 10 — หน้าสุด
    cout << "Back: "  << q.back()  << endl;  // 30 — ท้ายสุด

    q.pop();  // ลบ 10 ออก (ตัวหน้าสุด)

    cout << "Front after pop: " << q.front() << endl;  // 20

}
```

10. Pattern ที่ใช้บ่อย: BFS Style

```cpp
while (!q.empty()) {
    int x = q.front();
    q.pop();
    cout << x << endl;
}
```

ผลลัพธ์คือ element ถูกประมวลผล**ตามลำดับที่ใส่เข้าไป** (FIFO)
ตัวอย่าง use case ในการแข่งขัน:

- **BFS (Breadth-First Search)** — หา shortest path บน graph/grid
- **Multi-source BFS** — เช่น Rotting Oranges
- **Simulation** — จำลองระบบคิวตามลำดับเวลา

---

11. เปรียบเทียบ Stack vs Queue

โครงสร้าง	หลักการ	ใส่ข้อมูล	นำออก	อ่านค่า
Stack	LIFO (เข้าหลังออกก่อน)	push	pop	top
Queue	FIFO (เข้าก่อนออกก่อน)	push	pop	front

วิธีเลือกว่าจะใช้อันไหน:

- โจทย์ต้องการ **ย้อนกลับ / จัดการสิ่งล่าสุดก่อน** → **Stack**
- โจทย์ต้องการ **ประมวลผลตามลำดับ / สิ่งที่มาก่อนออกก่อน** → **Queue**

---

12. สรุปสิ่งสำคัญ

1. Stack และ Queue คือ **Abstract Data Type (ADT)** — กำหนดพฤติกรรม ไม่ใช่การ implement
2. STL implement ทั้งคู่เป็น **container adaptor** ที่ห่อหุ้ม deque ไว้ข้างใน
3. ทุก operation มี complexity **O(1)**
4. pop() ทั้งคู่ **ไม่คืนค่า** — ต้องอ่านก่อนด้วย top() หรือ front() แล้วค่อย pop()